React.js

Reusable Components in Web Development

Build Once • Reuse Anywhere • Develop Faster

Web Development Frameworks (ITUE203)

1. Introduction

React.js is a popular JavaScript library developed by Facebook for building user interfaces. It uses a component-based architecture, which allows developers to create reusable UI components. These components can be used multiple times across different parts of a website, making the development process faster, cleaner and easier to maintain.

React.js

2. Problem Statement

In traditional web development, the same UI elements (such as navbar, button, footer, etc.) are often written separately for each page.
This leads to:

- Code duplication
- More development time
- Difficulty in updating changes
- Inconsistent design across pages

There is a need for a solution that allows these common UI parts to be created once and reused across multiple pages.

3. How React Solves It

React uses a component-based architecture. Each part of the UI (like navbar, button, footer) is built as a separate component. These components can be reused in multiple pages without rewriting the code.

UI is divided into components

↓

Components are created once

↓

Reuse the same components across multiple pages

↓

Faster development, cleaner code, easy maintenance

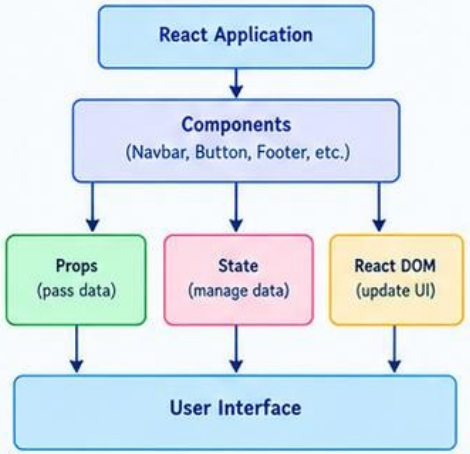
4. Component Reusability

Example: Navbar Component

The same navbar is used in 5 different pages of the website.



5. Architecture of React



6. Code Example (Navbar Component)

```jsx
// Navbar.js
import React from 'react';
import { Link } from 'react-router-dom';

const Navbar = () => {
  return (
    <nav className="navbar">
      <div className="logo">MyWebsite</div>
      <ul className="nav-links">
        <li><Link to="/">Home</Link></li>
        <li><Link to="/about">About</Link></li>
        <li><Link to="/services">Services</Link></li>
        <li><Link to="/blog">Blog</Link></li>
        <li><Link to="/contact">Contact</Link></li>
      </ul>
    </div>
  );
};
export default Navbar;
```

7. Example – 5 Pages with Same Navbar



8. Real-World Use Cases

- **E-commerce Websites** — Reusable product cards and navigation bars.
- **Dashboards & Admin Panels** — Common layout and UI components across sections.
- **Blogs and News Portals** — Shared header, footer and sidebar.
- **Business Websites** — Consistent design across all pages.

9. Advantages and Limitations

Advantages

- ✓ Reusability of components
- ✓ Faster development
- ✓ Cleaner and organized code
- ✓ Easy maintenance
- ✓ Consistent UI/UX

Limitations

- × Learning curve for beginners
- × Requires build tools (like Node.js)
- × Can be complex for very small projects
- × Frequent updates may require adaptation

10. Conclusion

Reusable components in React.js help developers build efficient, scalable and maintainable web applications. By creating common UI parts like navbar, button and footer once and reusing them across multiple pages, we save time, reduce code duplication and ensure a consistent user experience.

Same Component Many Pages Better Development

Name (Student 1): BUTANI YAKSH
Enrollment ID: 25DCE014

Name (Student 2): ANSH DABHI
Enrollment ID: 25DCE017